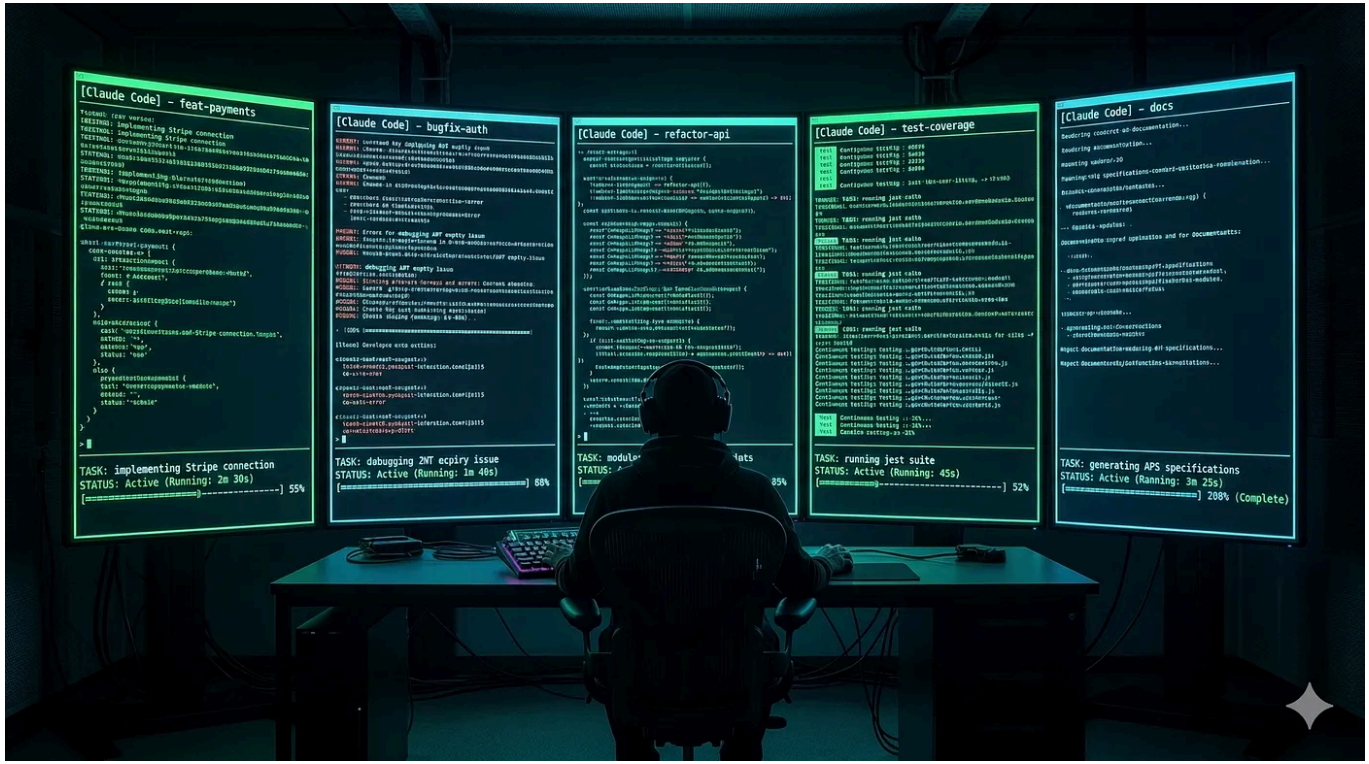

I Run 5 AI Agents in Parallel. Here's the Exact Setup.

Ayesha Mughal Follow 9 min read · Mar 7, 2026

176 6

Most developers use Claude Code like a calculator.

Type a question. Wait for the answer. Type the next question. Wait.

One task. One agent. One thread of progress.

Boris Cherny — the person who built Claude Code — doesn't work like that. He revealed his setup in a January thread that VentureBeat, Fortune, and InfoQ all covered the same week. The part that made developers stop scrolling:

"I run 5 Claudes in parallel in my terminal. I number my tabs 1–5, and use system notifications to know when a Claude needs input."

Five agents. Simultaneously. On the same repository.

While you're reviewing one agent's output, four others are already working.

Here's exactly how to copy his setup — including a February 2026 update that makes it dramatically simpler than it used to be.

🧠 The Mental Model Shift First

Before the commands, understand the idea — because it changes how you think about AI-assisted development entirely.

The old mental model: **you and Claude are having a conversation.**

One conversation. One thread. You wait for Claude to finish before you do the next thing.

Cherny's mental model: **you are a manager. Claude agents are your team.**

You hand out tasks. Agents work independently. You check in when they need a decision. You review output when they finish.

The bottleneck isn't how fast Claude generates tokens. The bottleneck is **your attention** — and attention can be split across 5 agents as easily as 1, as long as each agent is working on a separate, well-defined task.

That's the entire philosophy. Now let's build it.

⚠️ The Problem You'd Hit Without This Guide

You might be thinking: *"I'll just open 5 terminals and run `claude` in each one."*

Try it. Both agents will be working in the same directory, on the same files, on the same branch. Agent 1 edits `src/auth.ts`. Agent 2 simultaneously rewrites `src/auth.ts`. You end up with corrupted half-applied changes, lost work, and merge conflicts that make no logical sense.

This is why most people give up on parallel agents after one attempt. The solution is **git worktrees** — and as of February 21, 2026, Claude Code has this built in natively.

🌳 What Is a Git Worktree? (30-Second Explanation)

A git repository normally has one working directory — one set of files on disk. When you switch branches, the files change.

A **git worktree** is a second (or third, or fifth) working directory attached to the same repository. Each worktree has its own branch and its own independent files on disk — but they all share the same git history and remote connections.

```
your-repo/                          ← main worktree (your main branch)
.claude/worktrees/
  ├── feat-payments/                ← worktree 1 (own branch, own files)
  ├── bugfix-auth/                  ← worktree 2 (own branch, own files)
  └── refactor-api/                 ← worktree 3 (own branch, own files)
```

Agent 1 edits files in `feat-payments/`. Agent 2 edits files in `bugfix-auth/`. They are completely isolated. Zero conflicts. Zero interference.

This is the infrastructure that makes parallel agents actually work.

⚡ The New Way — One Command (February 2026 Update)

Before February 21, 2026, setting up worktrees required multiple manual git commands. Cherny's team shipped native worktree support directly into the Claude Code CLI. Now it's one flag.

Start Claude in an isolated worktree:

```
claude --worktree feature-payments
# shorthand:
claude -w feature-payments
```

That single command:

1. Creates `.claude/worktrees/feature-payments/` inside your repo
2. Creates a new branch `worktree-feature-payments`
3. Starts a Claude Code session scoped entirely to that isolated directory
4. Your main working tree is completely untouched

Don't want to name it? Let Claude generate a name:

```
claude --worktree
# Claude picks a random name automatically
```

For long-running tasks — add `--tmux` to run in the background:

```
claude --worktree refactor-api --tmux
# Launches in its own tmux session — survives terminal closes
```

Use `--tmux` for anything that'll run for more than a few minutes. The session stays alive even if you close the terminal. Check back later.

🖥️ Building Cherny's Exact 5-Agent Setup

Here's the step-by-step implementation. Five agents, five terminals, zero conflicts.

Step 1 — Add worktrees to .gitignore (do this once)

```
echo ".claude/worktrees/" >> .gitignore
```

Without this, worktree directories show up as untracked files in your main repo. Do it once, never think about it again.

Step 2 — Open 5 terminal tabs

In iTerm2, Terminal, or whatever you use — open 5 tabs. Name them 1, 2, 3, 4, 5 (double-click the tab name to rename). Cherny numbers his exactly like this. It sounds trivial. It makes context-switching instant.

Step 3 — Launch one agent per tab

```
# Tab 1 — new feature
claude -w feat-1024-login-redesign
# Tab 2 - production bug
claude -w fix-987-payment-timeout
# Tab 3 - long refactor (background, keeps running if you close terminal)
claude -w refactor-user-service --tmux
# Tab 4 - writing tests
claude -w test-coverage-auth-module
# Tab 5 - documentation
claude -w docs-api-endpoints
```

Each tab is now a fully isolated Claude Code agent working on its own branch, in its own directory, with no knowledge of the others.

Step 4 — Set up notifications so you know when they need you

This is the key that makes the whole system work. You can't watch 5 terminal tabs simultaneously. Notifications let you work on other things and get pulled back only when an agent is waiting for a decision.

Inside any Claude Code session:

```
/hooks
```

Select **Notification** from the event list. Select **+ Match all.** Claude will now send a desktop notification whenever it:

- Finishes a task and is ready for the next
- Needs your input or approval to continue
- Encounters something it's not sure about

Five agents working. You get notified when any one of them needs you. In between — you're free.

📋 What to Give Each Agent

Parallel agents only work if you give each one a **clear, self-contained task** that doesn't depend on what another agent is doing.

Good task for an agent:

> “Implement the forgot password flow in the auth module. It should send a reset email via Resend, generate a secure token stored in Redis with a 15-minute TTL, and validate on the reset page. Write tests.”

Bad task for an agent:

> “Continue what the other agent started on the auth refactor”

Each agent gets its own context window. They don’t know what the other agents are doing. Design tasks that can succeed in isolation.

Cherny’s actual task distribution across his 5 agents:

- Tab 1: Active feature development (the main thing he’s building)
- Tab 2: Running and fixing tests from the feature
- Tab 3: Code review and refactoring of recently landed code
- Tab 4: Debugging a specific reported issue
- Tab 5: Documentation or infrastructure work

Notice the pattern: tasks that naturally run in parallel. Tests don’t block features. Reviews don’t block bug fixes. Each lane moves independently.

🔧 The Setup for Each New Worktree

One gotcha that trips people up: **each worktree is a clean checkout.**

No `node_modules`. No virtual environments. No build artifacts. No `.env` file.

You need to initialize each worktree before asking Claude to build or test:

```
# After launching claude -w feat-1024-payments
# Claude will land in the new worktree directory
# Tell it to set up first:
```

Inside the Claude session:

```
Before starting, run npm install to set up this worktree's dependencies.
Then implement: [your task here]
```

Or add this to your CLAUDE.md so it happens automatically:

```
## Worktree Setup
When starting a session in a new worktree (.claude/worktrees/),
always run `npm install` before doing anything else.
Then copy `.env.example` to `.env` and ask me for any missing values.
```

One rule in CLAUDE.md — every agent in every worktree handles setup automatically.

🌐 The Browser Sessions Too — Cherny's Full Setup

The 5 terminal agents are only half of it. Cherny's full setup:

- **5 terminal agents** (isolated worktrees, numbered tabs)
- **5–10 browser sessions** on claude.ai/code
- **Mobile sessions** he starts in the morning and checks on later

The browser sessions handle things that don't need local file access — research, planning, explaining architecture decisions, writing docs or communications.

The `--teleport` command lets him hand off a local session to the browser and back:

```
# Hand off current terminal session to browser
claude --teleport
# Claude gives you a URL — open it on any device
```

He starts complex tasks on his phone during morning commute. By the time he's at his desk, agents have been working for 40 minutes already.

🤝 Subagents in Worktrees — Going Even Further

The worktree isolation extends beyond your own sessions. Claude's internal subagents can use worktrees too.

When Claude spawns a subagent to handle a subtask, you can tell it to work in isolation:

```
Use worktrees for your subagents when working on this migration.
```

Or configure it permanently in a custom subagent file:

`.claude/agents/migration-worker.md`

```
---
isolation: worktree
description: Handles database migrations in isolated worktrees.
  Use for any schema changes that could conflict with active development.
---
When running database migrations:
1. Create the migration file following the naming convention
2. Write both up and down migrations
3. Run the migration against the test database
4. Verify with a rollback test
5. Commit and report results
```

Every time this agent runs, it automatically gets its own worktree. No flag needed. No manual setup.

🧹 Cleanup — When You're Done

When an agent finishes its task, Claude handles cleanup automatically:

- **No changes made:** Worktree and branch are removed automatically when the session ends

- **Changes exist:** Claude asks — keep the worktree for later, or remove it?

Manual cleanup when needed:

```
# See all active worktrees
git worktree list
# Remove a specific worktree
git worktree remove .claude/worktrees/feat-payments
# Clean up stale metadata if a worktree was force-deleted
git worktree prune
```

Keep your worktree list clean. More than 8–10 active worktrees gets hard to track.

📊 The Honest Reality Check

Running 5 parallel agents on **a subscription plan** (like Claude Pro) works fine — each session uses its usage allowance independently, but you're not billed per API token.

If you're running Claude Code via the **API directly**, parallel agents multiply your costs. Five agents running simultaneously burns roughly 5x the tokens

of one agent. Teams at incident.io who run 4–5 parallel agents daily report dramatically faster delivery — but they budget for it.

For the free/cheap setup from this series: running 3 parallel agents via OpenRouter's free tier uses 3x your daily limit. With 200 free requests/day per model, spread across 3 agents you get ~66 requests per agent per day. Enough for focused tasks — not unlimited.

Start with 2 parallel agents. Get comfortable with the workflow. Then scale up when you know exactly what you're buying with the extra compute.

📋 Quick Reference — The Commands You'll Actually Use

```
# Start agent in named worktree
claude --worktree task-name
claude -w task-name              # shorthand
# Start agent in background (survives terminal close)
claude -w task-name --tmux
# Let Claude auto-name the worktree
claude --worktree
# Set up notifications for all agents
# Inside any Claude session:
/hooks → Notification → Match all
# See all active worktrees
git worktree list
# Clean up a worktree
```

```
git worktree remove .claude/worktrees/task-name
# Teleport session to browser
claude --teleport
```

🎯 Start Today — The 10-Minute Version

You don't need 5 agents on day one. Start with 2.

```
# Add to .gitignore first
echo ".claude/worktrees/" >> .gitignore
# Open 2 terminal tabs
# Tab 1:
claude -w main-feature
# Tab 2:
claude -w background-task --tmux
```

Give Tab 1 the thing you're actively building. Give Tab 2 something that can run unattended — test coverage, documentation, a refactor you've been putting off.

Set up notifications. Check Tab 2 when it pings you.

That's it. That's the whole system. Add agents as you get comfortable with the rhythm.

One question before you go:

What's the one task in your workflow you'd hand to a background agent first — tests, docs, refactoring, or something else entirely? Drop it in the comments. I'm genuinely curious, and it might become the next post.

*If this saved you an hour of googling, hit **Follow** — Day 7 drops tomorrow and it's the one that ties everything together: Hooks, the feature that runs automatically without you typing anything.*

Next up: Hooks — how to make Claude automatically format your code, run tests, and send you notifications without typing a single command

Claude Code | AI | Productivity | Programming | Generative Ai Tools